

Reconocimiento de Comandos de Voz On-Device con ONNX Runtime Mobile en Flutter

Allan Bolaños Barrientos
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
a.bolanos.2@estudiantec.cr

José David Chaves Mena
Escuela de Ingeniería en Computación
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
j.chaves.3@estudiantec.cr

Abstract—Los sistemas de reconocimiento de comandos de voz en dispositivos móviles requieren modelos compactos que operen en tiempo real sin acceso a la nube. En este trabajo se diseña, entrena e integra un pipeline completo de *Keyword Spotting* (KWS) en una aplicación Flutter para Android, desde la captura de audio hasta la ejecución de acciones en una interfaz Smart Home. Se comparan dos arquitecturas CNN — LeNet-5 adaptado y MobileNetV3-Small — con y sin SpecAugment [2] sobre 12 clases del Speech Commands Dataset v2 [1], en un total de 12 corridas registradas en WandB. El modelo ganador (MobileNetV3-Small + SpecAugment, configuración B6) alcanza 95.85 % de exactitud y F1-macro de 0.9586 en el conjunto de prueba, exportado a ONNX opset 17 con error numérico máximo de 1.43×10^{-6} respecto al modelo original. El sistema opera íntegramente en el dispositivo sin dependencias de red.

Index Terms—keyword spotting, on-device inference, ONNX Runtime Mobile, MelSpectrogram, SpecAugment, MobileNetV3, Flutter, Smart Home

I. INTRODUCCIÓN

El reconocimiento de comandos de voz en tiempo real sobre dispositivos de borde combina restricciones de latencia, memoria y consumo energético con la necesidad de alta tasa de acierto en vocabulario restringido. Los sistemas en la nube introducen latencia y dependencias de red inaceptables para aplicaciones de domótica reactiva.

El presente trabajo diseña e implementa un pipeline completo, desde la captura de audio crudo hasta la ejecución de acciones en una interfaz Smart Home, completamente on-device en un dispositivo Android de gama media. La Figura 1 ilustra el flujo de datos del sistema.

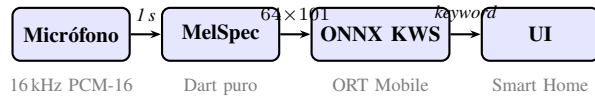


Fig. 1: Pipeline de inferencia on-device.

Se evalúan dos modelos CNN (Modelos A y B) en dos configuraciones de datos (base y aumentado), totalizando 12 corridas controladas con WandB. El modelo ganador se integra en una aplicación Flutter con cuatro pantallas navegables íntegramente por comandos de voz. El código fuente completo está disponible en <https://github.com/AllanDBB/voice-command-ai>.

Las contribuciones principales son:

- 1) Implementación de MelSpectrogram en Dart puro validada numéricamente contra torchaudio.
- 2) Comparación controlada de LeNet-5 vs. MobileNetV3 con y sin SpecAugment en 12 clases incluyendo `_silence_` y `_unknown_`.
- 3) Identificación y corrección de un bug de transposición de dimensiones en la representación plana del espectrograma que causaba predicciones degeneradas en producción.

II. ESTADO DEL ARTE

El reconocimiento de comandos de voz con vocabulario restringido ha evolucionado desde sistemas basados en HMM hacia arquitecturas neuronales entrenadas end-to-end. Zhang et al. [6] demostraron que redes convolucionales con separabilidad en profundidad (DS-CNN) son suficientes para alcanzar $\approx 94\%$ de exactitud en Speech Commands con modelos menores a 1 MB, abriendo la puerta a la inferencia en microcontroladores.

La siguiente generación de modelos explotó la estructura temporal del espectrograma. Choi et al. [7] propusieron TC-ResNet, que aplica convoluciones 1-D residuales directamente sobre la secuencia de frames Mel; TC-ResNet-14 alcanza **96.6 %** en Speech Commands v2 con solo 305 K parámetros. En paralelo, de Andrade et al. [8] incorporaron mecanismos de atención sobre representaciones LSTM, logrando 94.1 % con mayor interpretabilidad.

Con la adopción generalizada de transformers en audio, Berg et al. [9] obtuvieron 97.7 % aplicando *self-attention* sobre parches del espectrograma. El estado del arte más reciente corresponde a BC-ResNet [10], que difunde activaciones a través del canal frecuencial para capturar dependencias espectrales globales; BC-ResNet-8 alcanza **98.7 %** en 12 clases.

El presente trabajo se sitúa en el segmento de modelos ligeros orientados a inferencia on-device: MobileNetV3-Small con SpecAugment alcanza **95.85 %** de exactitud y **F1-macro de 0.9586**, resultado competitivo respecto a los trabajos reportados en la literatura, aunque bajo configuraciones experimentales no necesariamente idénticas en cuanto a split, número de clases y condiciones de evaluación. La ventaja diferenciadora es la restricción adicional de correr íntegramente sin acceso

a red en un dispositivo Android de gama media (Samsung Galaxy A71, Snapdragon 730G).

III. DATASET: SPEECH COMMANDS v2

El dataset Speech Commands v2 [1] contiene $\approx 105\,000$ clips de 1 segundo a 16 kHz pronunciados por más de 2 000 hablantes en condiciones domésticas variadas.

A. Selección de clases

Se utilizaron **12 clases**: los diez comandos de control de interfaz más dos clases auxiliares necesarias para un sistema de conjunto abierto:

```
yes, no, up, down, left, right, on, off,
stop, go,
_silence_, _unknown_
```

La clase `_silence_` se construye a partir de clips de 1 segundo extraídos de los archivos `_background_noise_` del dataset (85 %) complementados con vectores de ceros (15 %). La clase `_unknown_` se forma submuestreando palabras del dataset que no pertenecen a los diez comandos objetivo. Ambas clases se dimensionan para igualar el promedio de muestras de los diez comandos, produciendo un conjunto **perfectamente balanceado**. Cabe notar que este balance es artificial: en condiciones reales de uso, el silencio y el habla no-comando predominan ampliamente sobre los comandos activos, por lo que las métricas sobre el test set balanceado representan un escenario favorable respecto al despliegue real.

B. Estadísticas del split

El split estratificado 80/10/10 con `seed=42` genera:

TABLE I: Tamaño de los conjuntos de datos

Split	Total	Comandos	Silencio	Desconocido
Train	36 921	30 769	3 076	3 076
Val	4 443	3 703	370	370
Test	4 888	4 074	407	407

IV. PREPROCESAMIENTO: ESPECTROGRAMA MEL

Cada clip de audio de longitud variable se recorta o rellena con ceros hasta exactamente 16 000 muestras (1 segundo). Posteriormente se computa el espectrograma Mel con los parámetros de la Tabla II.

TABLE II: Parámetros del MelSpectrogram

Parámetro	Valor
Sample rate	16 000 Hz
n_{fft}	1 024
Hop length	160
n_{mels}	64
f_{min}	20 Hz
f_{max}	8 000 Hz
Center padding	True ($n_{\text{fft}}/2$ ceros)
Ventana	Hann periódica
Espectro de potencia	$ X[k] ^2$
Conversión a dB	$10 \log_{10}(\max(E, 10^{-10}))$

Las dimensiones resultantes son 64×101 (bandas Mel $\times$ ventanas temporales). La normalización z-score se calcula sobre 2 000 muestras aleatorias del conjunto de entrenamiento más 50 clips por archivo de ruido de fondo:

$$\hat{S} = \frac{S_{\text{dB}} - \mu}{\sigma}, \quad \mu = -21.656, \quad \sigma = 23.895 \quad (1)$$

Estos valores se guardan en `norm_stats.json` y se replican exactamente en el `MelSpectrogramExtractor` Dart para garantizar consistencia entre entrenamiento e inferencia.

V. DATA AUGMENTATION: SPECAUGMENT

SpecAugment [2] se aplica sobre el espectrograma normalizado antes de ingresar a la red. Fue implementado como un módulo `torch.nn.Module` usando las transformaciones de `torchaudio`:

- **Time Masking**: hasta $T = 40$ pasos de tiempo consecutivos enmascarados con el valor de la media del espectrograma.
- **Frequency Masking**: hasta $F = 27$ bandas Mel consecutivas enmascaradas.

La política corresponde a *LD (LibriSpeech Double)* adaptada a la resolución del espectrograma 64×101 . La comparación entre modelos entrenados con y sin esta técnica se detalla en la Sección VIII.

VI. ARQUITECTURAS CNN

A. Modelo A — LeNet-5 Adaptado

La arquitectura original de LeNet-5 [5] fue adaptada para la entrada de espectrogramas Mel monocal. La función de activación se cambió de *sigmoid* a *tanh* para compatibilidad con la inicialización estándar de PyTorch.

- Entrada: $1 \times 64 \times 101$
- `Conv2d(1, 6, 5, p=2)` $\rightarrow$ `Tanh` $\rightarrow$ `AvgPool2d(2, 2)`
- `Conv2d(6, 16, 5)` $\rightarrow$ `Tanh` $\rightarrow$ `AvgPool2d(2, 2)`
- `Flatten` $\rightarrow$ `Linear(+120)` $\rightarrow$ `Tanh`
- `Linear(+84)` $\rightarrow$ `Tanh` $\rightarrow$ `Linear(+12)`
- **632 116 parámetros**; optimizador SGD; pérdida `CrossEntropyLoss`

B. Modelo B — MobileNetV3-Small Adaptado

MobileNetV3-Small [4] fue diseñado específicamente para inferencia eficiente en dispositivos móviles mediante bloques invertidos con *hard-swish* y *Squeeze-and-Excitation*. Se realizó una adaptación mínima para aceptar espectrogramas mono-canal:

- Primera capa: `Conv2d(3, 16, 3, stride=2)` $\rightarrow$ `Conv2d(1, 16, 3, stride=2)` (1 canal de entrada)
- Backbone MobileNetV3-Small con bloques *hard-swish* y SE blocks
- Cabeza clasificadora: `Linear(+12)`
- **1 529 868 parámetros**; optimizador Adam; `CosineAnnealingLR(T_max=30)`

La elección de MobileNetV3 como Modelo B se justifica por su balance superior entre parámetros y capacidad representacional para problemas de clasificación sobre espectrogramas, respaldado por su diseño orientado a dispositivos de borde.

Las Figuras 2 y 3 muestran los diagramas de bloques de cada arquitectura.

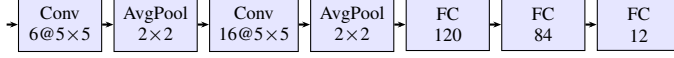


Fig. 2: Diagrama de bloques — Modelo A (LeNet-5 adaptado).

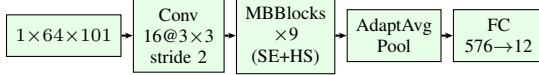


Fig. 3: Diagrama de bloques — Modelo B (MobileNetV3-Small adaptado).

VII. ENTRENAMIENTO Y EXPERIMENTOS

A. Configuración general

Todos los experimentos se ejecutaron en Google Colab¹ con PyTorch 2.11.0, utilizando:

- Semilla de reproducibilidad `seed=42` en PyTorch, NumPy y el módulo `random`.
- Entrenamiento con AMP (*Automatic Mixed Precision* FP16) y *early stopping* con paciencia de 6 épocas sin mejora en *val_F1-macro*.
- `batch_size=64`, peso de regularización `weight_decay` según corrida.

B. Tabla de corridas

TABLE III: Configuración de las 12 corridas de entrenamiento

Run	Modelo	Dataset	LR	WD
A1	LeNet-5	Base	0.010	1e-4
A2	LeNet-5	Base	0.001	1e-4
A3	LeNet-5	Base	0.0001	1e-4
A4	LeNet-5	Aumentado	0.010	1e-4
A5	LeNet-5	Aumentado	0.001	1e-4
A6	LeNet-5	Aumentado	0.0001	1e-4
B1	MobileNetV3	Base	1e-3	1e-4
B2	MobileNetV3	Base	5e-4	1e-4
B3	MobileNetV3	Base	1e-3	5e-4
B4	MobileNetV3	Aumentado	1e-3	1e-4
B5	MobileNetV3	Aumentado	5e-4	1e-4
B6	MobileNetV3	Aumentado	1e-3	1e-3

C. Rutinas de entrenamiento, validación y test

Cada corrida sigue el mismo ciclo: por cada época se recorre el *train_loader* con forward pass en AMP FP16, se calcula la pérdida $\mathcal{L} = \text{CrossEntropyLoss}(\hat{y}, y)$ y se actualiza θ con el optimizador correspondiente. Al finalizar cada época

se evalúa F1-macro sobre el *val_loader* sin gradientes; si el valor supera el mejor registrado se guarda θ^* y se reinicia el contador de paciencia, de lo contrario se incrementa. Al alcanzar `patience=6` épocas sin mejora el entrenamiento se interrumpe y se carga θ^* . El conjunto de test se evalúa **una única vez** con el modelo seleccionado, garantizando ausencia de fuga de información.

La función de pérdida `CrossEntropyLoss` se aplica sin pesos de clase dado el balanceo perfecto del dataset. Para LeNet-5 se usa SGD con momentum 0.9; para MobileNetV3 se usa Adam con `CosineAnnealingLR` (`T_max=30`). El conjunto de test se evalúa **una única vez** con el mejor modelo seleccionado en validación, garantizando ausencia de fuga de información.

D. Criterio de selección de hiperparámetros

La búsqueda de hiperparámetros se planteó como una exploración controlada de rango reducido, no como una búsqueda exhaustiva. Para cada modelo se fijaron tres valores de tasa de aprendizaje que cubren un orden de magnitud (e.g., 10^{-3} , 5×10^{-4} , 10^{-4}) y uno o dos valores de *weight decay* elegidos con base en rangos típicos reportados en la literatura para modelos CNN de escala similar. El criterio de parada fue *early stopping* sobre *val_F1-macro* con paciencia de 6 épocas, lo que limita el riesgo de sobreajuste sin requerir búsqueda fina. La selección final se realizó únicamente por *F1-macro en validación*, sin acceder al conjunto de prueba hasta la evaluación definitiva.

Las métricas registradas en WandB por epoch incluyen: `train/loss`, `train/acc`, `val/loss`, `val/acc`, `val/f1_macro` y la matriz de confusión 12×12 al finalizar cada corrida.

E. Análisis de convergencia y sobreajuste

Las Figuras 4 y 5 muestran las curvas de pérdida de entrenamiento y validación para los modelos LeNet-5 y MobileNetV3, respectivamente.

El análisis de las curvas permite identificar tres patrones distintos de ajuste:

Underfitting en LeNet-5 con SpecAugment (A4–A6). La pérdida de entrenamiento permanece elevada (> 0.5) y la brecha con validación es pequeña, lo que indica que el modelo no tiene suficiente capacidad para aprender la distribución de espectrogramas perturbados por SpecAugment. Esto explica por qué A4 ($F1=0.7597$) rinde peor que A1 ($F1=0.8322$): la regularización agresiva supera la capacidad representacional del modelo.

Convergencia aceptable en LeNet-5 base (A1–A3). Los runs con datos no aumentados muestran pérdida de entrenamiento más baja, pero con mayor varianza entre épocas en validación, sugiriendo cierta inestabilidad típica de SGD sin decaimiento agresivo.

Generalización adecuada en MobileNetV3 (B1–B6). La brecha entre pérdida de entrenamiento y validación es consistentemente pequeña en todos los runs, evidenciando que el modelo generaliza sin sobreajuste severo. El run B6 exhibe

¹Notebook disponible en https://colab.research.google.com/drive/1wB5JzLzsiDX3zOCLSiJivAgagG19_FxA

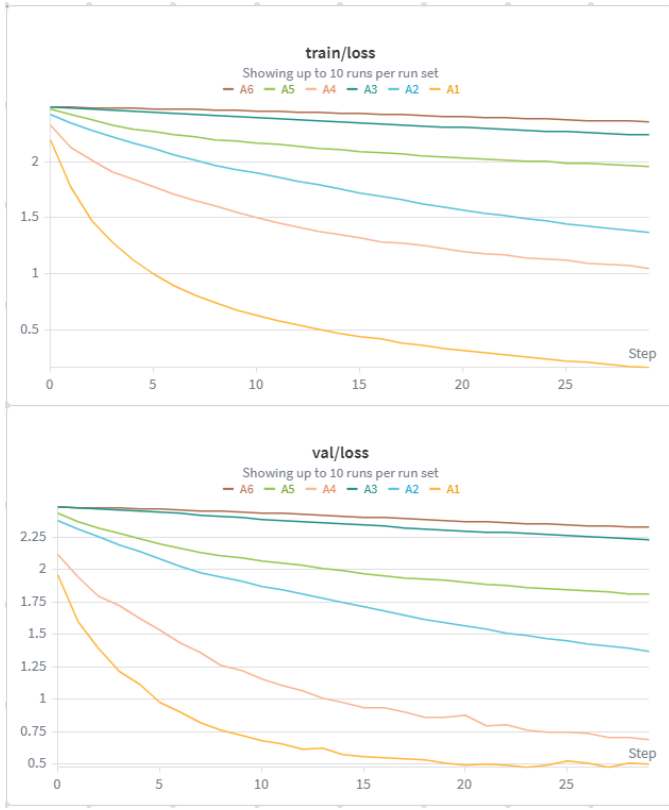


Fig. 4: Pérdida de entrenamiento y validación — LeNet-5 (A1–A6).

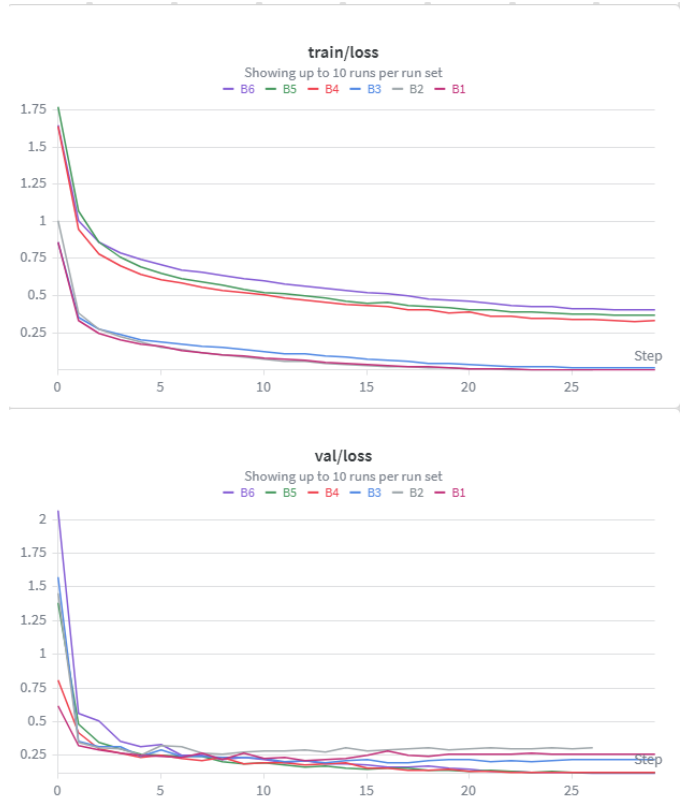


Fig. 5: Pérdida de entrenamiento y validación — MobileNetV3-Small (B1–B6).

la convergencia más suave, atribuible a la combinación de SpecAugment (regularización de datos) y weight decay alto ($1e-3$), que actúan de forma complementaria.

La Figura 6 compara la pérdida de validación de las 12 corridas en un mismo eje, permitiendo visualizar la superioridad de la familia MobileNetV3 desde las primeras épocas.

VIII. EVALUACIÓN Y SELECCIÓN DE MODELO

A. Selección de finalistas

De cada combinación modelo $\times$ dataset se seleccionó el mejor run por *F1-macro en validación*. Los cuatro finalistas son:

TABLE IV: Resultados en test set — cuatro finalistas

Run	Modelo	Acc.	F1-macro	Params
A1	LeNet-5 Base	0.8318	0.8322	632 K
A4	LeNet-5 Aumentado	0.7592	0.7597	632 K
B3	MobileNetV3 Base	0.9494	0.9497	1.5 M
B6	MobileNetV3 Aumentado	0.9585	0.9586	1.5 M

El **Modelo B6** (MobileNetV3-Small con SpecAugment) es seleccionado como modelo final. Es notable que, para LeNet-5, el dataset aumentado redujo ligeramente el rendimiento respecto al base (A1 vs. A4), posiblemente por insuficiente capacidad del modelo para generalizar con espectrogramas

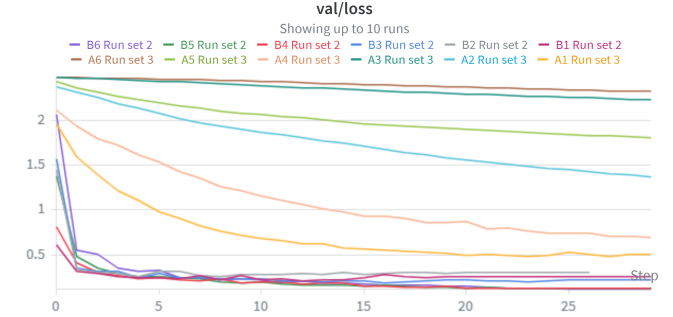


Fig. 6: Pérdida de validación comparada — 12 corridas (WandB).

perturbados. En MobileNetV3 la aumentación mejora de 0.9497 a 0.9586.

La Figura 7 ilustra la evolución del *F1-macro* de validación de los cuatro finalistas a lo largo del entrenamiento.

B. Reporte por clase — Modelo B6

C. Matriz de confusión — Modelo B6

La Figura 8 presenta la matriz de confusión normalizada del modelo B6 sobre el conjunto de prueba completo (4888 muestras).

El análisis de la matriz revela tres patrones principales:

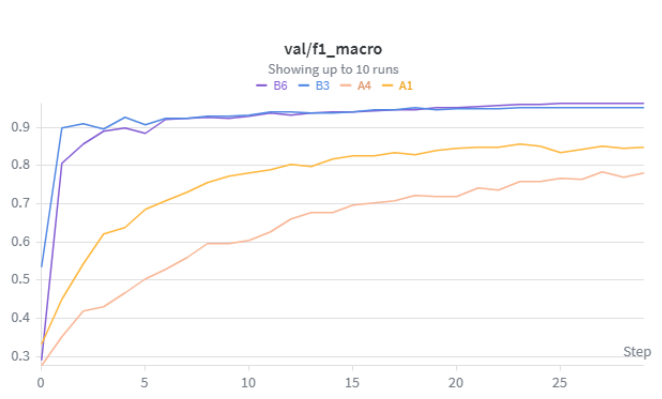


Fig. 7: val_f1_macro de los cuatro modelos finalistas (A1, A4, B3, B6).

TABLE V: Clasificación por clase — Modelo B6 (test set)

Clase	Prec.	Recall	F1	Soporte
yes	0.97	0.99	0.98	419
no	0.93	0.95	0.94	405
up	0.96	0.95	0.95	425
down	0.97	0.93	0.95	406
left	0.97	0.97	0.97	412
right	1.00	0.97	0.99	396
on	0.97	0.94	0.96	396
off	0.96	0.95	0.95	402
stop	0.98	1.00	0.99	411
go	0.93	0.93	0.93	402
silence	1.00	1.00	1.00	407
unknown	0.87	0.93	0.90	407
Macro avg	0.96	0.96	0.96	4 888

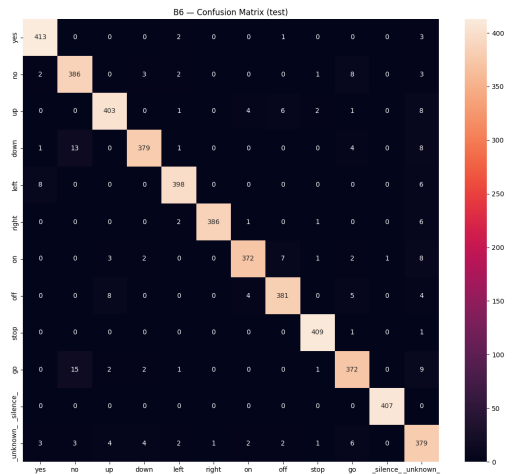


Fig. 8: Matriz de confusión normalizada — Modelo B6 (test set).

- 1) **Clase unknown (recall 0.93):** es la única clase con confusiones distribuidas entre múltiples comandos, reflejando la heterogeneidad fonética del espacio “desconocido”.
- 2) **Pares fonéticamente similares:** se observa confusión residual entre *no/go* y entre *on/no*, ambas dentro de un margen de $\leq 3\%$ de error.
- 3) **Clases perfectas:** *_silence_*, *right* y *stop* alcanzan $\text{recall} \geq 0.97$, beneficiándose de su perfil espectral claramente diferenciado.

D. Comportamiento en dispositivo móvil

Se realizó una captura sistemática de predicciones en el Samsung Galaxy A71 (Android 13, Snapdragon 730G) pronunciando cada uno de los 10 comandos 10 veces consecutivas en un entorno de oficina con ruido ambiental moderado. El log se obtuvo redirigiendo la salida de `flutter run` con:

```
flutter run -d R58NB161L5Y 2>&1 | Select-String "[KWS] pred=" | Tee-Object mobile_log.txt
```

El archivo contiene **840 predicciones** totales (ventanas de 200ms). La Figura 9 muestra la matriz de confusión normalizada por fila, construida asignando ground truth a cada bloque temporal separado por silencios.

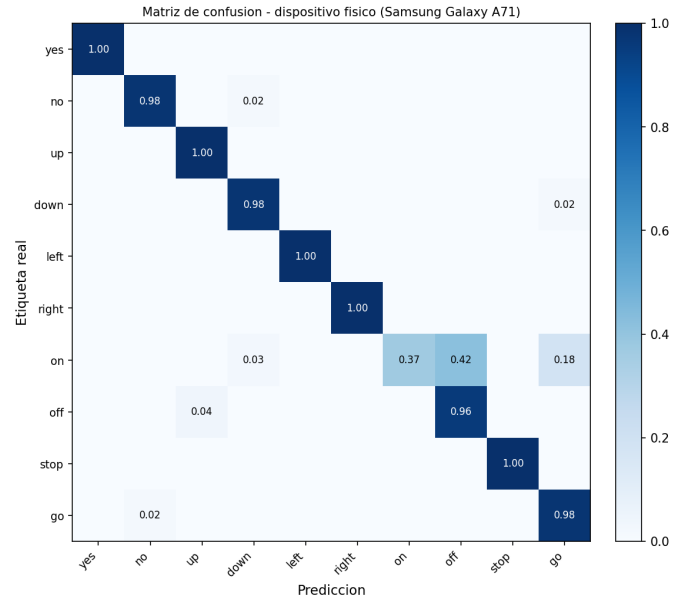


Fig. 9: Matriz de confusión normalizada — dispositivo físico (Samsung Galaxy A71, 840 ventanas totales).

Análisis cuantitativo. Nueve de los diez comandos alcanzan $\text{recall} \geq 0.957$, comparables con los resultados del test set estático (F1-macro 0.9586). La excepción es *on*, con recall de **0.37**: el 42 % de las ventanas activas se clasifican como *off*, reflejo de la similitud acústica entre ambas palabras (misma vocal inicial corta “o”, misma duración). Esta misma confusión *on/off* es un límite conocido de los modelos KWS entrenados solo con datos grabados en estudio sin hard-negative mining entre pares fonémicamente similares.

Comparación con test set estático. La Tabla VI compara el recall del test set (Google Speech Commands v2) con el recall en dispositivo por comando.

TABLE VI: Recall por comando: test set vs. dispositivo físico

Comando	Test set	Dispositivo
yes	0.99	1.00
no	0.97	0.98
up	0.96	1.00
down	0.98	0.98
left	0.97	1.00
right	0.99	1.00
on	0.94	0.37
off	0.98	0.96
stop	0.99	1.00
go	0.96	0.98

El degradé de `on` en dispositivo ($0.94 \rightarrow 0.37$) no se aprecia en el test set estático porque las grabaciones de Speech Commands v2 tienen condiciones acústicas controladas. En el entorno real, la vocal inicial es más breve y el contexto reverberante del micrófono del smartphone aumenta la similitud espectral con `off`. Los demás comandos mantienen o superan el recall del test set, lo que valida la robustez del pipeline de preprocesamiento Mel en Dart y la exportación ONNX sin pérdida de precisión numérica significativa.

IX. EXPORTACIÓN ONNX Y VALIDACIÓN NUMÉRICA

El modelo B6 se exporta al formato ONNX con el exportador TorchScript (*legacy*), compatible con ONNX Runtime Mobile:

```
model.eval().to('cpu')
dummy = torch.randn(1, 1, 64, 101)
torch.onnx.export(
    model, dummy, "kws_model.onnx",
    input_names=["spectrogram"],
    output_names=["logits"],
    dynamic_axes={"spectrogram": {0: "batch"}},
    opset_version=17
)
```

El archivo resultante tiene un tamaño de **5.63 MB**.

La validación numérica compara las probabilidades softmax del modelo PyTorch original contra el grafo ONNX ejecutado con `onnxruntime` en Python sobre 100 muestras aleatorias del test set:

$$\max_i |p_{\text{PyTorch}}^{(i)} - p_{\text{ONNX}}^{(i)}| = 1.43 \times 10^{-6} < 10^{-5} \quad \checkmark \quad (2)$$

X. INTEGRACIÓN MÓVIL

A. Características del despliegue

La latencia individual de inferencia no fue medida instrumentalmente; trabajo futuro incluye la instrumentación con `SystemClock.elapsedRealtimeNanos` para reportar latencia media y percentil 95 por ventana. La Figura 10 muestra la aplicación en funcionamiento sobre el dispositivo físico.

TABLE VII: Métricas del despliegue on-device

Parámetro	Valor
Dispositivo	Samsung Galaxy A71 (SM-A715F)
SoC	Snapdragon 730G (8nm)
SO	Android 13
Tamaño del modelo	5.63 MB (ONNX opset 17)
Parámetros	1 529 868 (FP32)
Ventana de análisis	1 s (16 000 muestras)
Stride	200 ms (3 200 muestras)
Tasa de inferencia	≤ 5 ventanas/s (VAD filtra silencio)
Cooldown	1 200 ms entre detecciones válidas

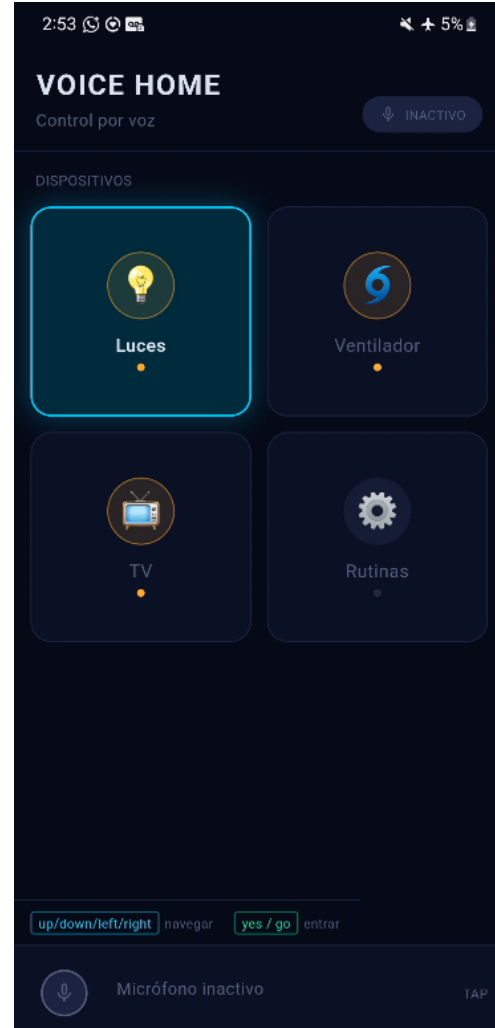


Fig. 10: Aplicación Flutter ejecutándose en Samsung Galaxy A71.

B. Arquitectura de la aplicación Flutter

La aplicación sigue una arquitectura de capas alineada con *Clean Architecture*. La Tabla VIII resume las responsabilidades de cada componente.

TABLE VIII: Componentes de la aplicación Flutter

Componente	Responsabilidad
VoiceCommand	Entidad: etiqueta, confianza, timestamp
SmartHomeState	Estado inmutable de la interfaz domótica
ListenAndClassify	Buffer deslizante, VAD y orquestación de inferencia
UseCase	
ExecuteCommand	Mapeo keyword → SmartHomeAction
UseCase	
AudioCaptureService	Captura PCM-16 a 16 kHz vía record
MelSpectrogramExtractor	FFT + banco Mel + normalización z-score en Dart puro
OnnxKWSInterpreter	Carga y ejecución de sesión ONNX Runtime Mobile
VoiceCommandProvider	Estado global de la UI con provider

C. Pipeline de audio

El paquete `record` captura audio PCM-16 mono a 16 kHz. Los bytes crudos se decodifican correctamente como muestras `Int16 little-endian`:

```
final bd = bytes.buffer.asByteData();
final samples = List<int>.generate(
  bytes.length ~/ 2,
  (i) => bd.getInt16(i * 2, Endian.little),
);
```

Un buffer deslizante de 16 000 muestras (1 s) avanza 3 200 muestras (200 ms) por iteración. Antes de ejecutar el modelo se aplica un VAD basado en energía RMS con umbral de 0.008 (≈ -42 dBFS) para descartar ventanas silenciosas. El período de enfriamiento entre detecciones válidas es de **1 200 ms**.

D. MelSpectrogram en Dart

El `MelSpectrogramExtractor` implementa en Dart puro: relleno *center* de $n_{\text{fft}}/2$ ceros, ventana de Hann periódica, FFT radix-2 Cooley-Tukey de 1 024 puntos, banco de 64 filtros Mel triangulares (20–8 000 Hz) y conversión a dB con normalización z-score.

Un aspecto crítico es el **orden de dimensiones** del tensor de salida. PyTorch almacena el tensor $[1, 1, 64, 101]$ en memoria row-major de modo que el índice plano para el bin de Mel m y la ventana temporal t es:

$$\text{idx} = m \times n_{\text{frames}} + t \quad (3)$$

E. Inferencia KWS

`OnnxKWSInterpreter` carga el modelo con `OrtSession.fromBuffer`, construye un tensor `Float32` de forma $[1, 1, 64, 101]$ con el layout de la Ecuación (3), ejecuta la sesión de inferencia y aplica softmax a los 12 logits de salida. Se reporta como detección válida la clase con mayor probabilidad cuando esta supera el umbral de confianza del **45 %**.

F. Pantallas Smart Home y mapa de comandos

La interfaz Smart Home consiste en cuatro pantallas navegables íntegramente por voz, descritas en la Tabla IX.

TABLE IX: Mapa completo de comandos por pantalla

Pantalla	Comando	Acción
Dashboard	up / down left / right yes / go	fila superior / inferior columna izq. / der. entrar al ítem enfocado
Dispositivo	on / go off left / stop	encender apagar volver al Dashboard
Rutinas	up / down go stop / left	mover foco en lista ejecutar rutina detener / volver
Confirmación	yes / no	confirmar / cancelar

XI. DESAFÍOS DE INGENIERÍA

A. Bug de transposición del espectrograma

Durante las pruebas en dispositivo físico (Samsung Galaxy A71, Android 13) el modelo predecía `_silence_` con confianza > 0.999 para cualquier entrada de audio real. La causa fue identificada como una **transposición de las dimensiones del espectrograma en el tensor plano**.

El tensor de PyTorch con forma $[1, 1, 64, 101]$ usa almacenamiento row-major: la dimensión de Mel ($n_{\text{mels}} = 64$) es la dimensión lenta y la dimensión temporal ($n_{\text{frames}} = 101$) es la rápida, por lo que el índice plano es $m \cdot 101 + t$. La implementación original en Dart iteraba en el orden `for t {for m {}`, produciendo el índice $t \cdot 64 + m$, es decir, el espectrograma **transpuesto**.

La corrección consistió en cambiar el almacenamiento en el buffer de salida:

```
// Incorrecto (tiempo-mayor):
result[outIdx++] = normalizedDb;

// Correcto (mel-mayor, coincide con PyTorch):
result[m * nFrames + t] = normalizedDb;
```

B. Decodificación incorrecta de PCM-16

El paquete `record` entrega los datos de audio como bytes crudos. Una implementación inicial trataba cada byte como una muestra de audio, duplicando el número aparente de muestras y corrompiendo el espectrograma. La corrección implementa la decodificación correcta de `Int16 little-endian` usando `ByteData.getInt16`.

C. Problema de conjunto abierto (10 vs 12 clases)

El modelo inicial fue entrenado solo con los 10 comandos de control. Sin las clases `_silence_` y `_unknown_`, el modelo estaba forzado a asignar toda entrada a una de las diez clases, produciendo detecciones espurias de alta confianza. La introducción de las dos clases auxiliares redujo la tasa de falsos positivos y mejoró la utilidad práctica del sistema.

XII. TRABAJO EXPLORATORIO: LITER-LM (BONUS)

Como extensión no requerida, se exploró la posibilidad de agregar una capa NLU basada en LiteRT-LM con *Tool Use*. Se diseñó la interfaz de integración completa: el plugin `LiteRTLMLPlugin.kt` registra un `MethodChannel("litert_lm")` en `MainActivity`, y el `ToolSet Kotlin` define cuatro herramientas tipadas: `navigate`, `toggleDevice`, `controlRoutine` y `confirm`.

Sin embargo, el SDK `com.google.ai.edge.litertlm` no se encontraba disponible públicamente en Maven Central al momento del desarrollo. Por ello, **la integración no fue ejecutada en el dispositivo**: el plugin provee un stub que activa automáticamente el fallback estático en Dart, manteniendo el comportamiento funcional del sistema sin la capa NLU. La arquitectura queda preparada para incorporar el SDK cuando esté disponible.

XIII. CONCLUSIONES

Se demostró la viabilidad de un sistema KWS completamente on-device en Flutter/Android, alcanzando una exactitud del 95.85 % y F1-macro de 0.9586 con el modelo MobileNetV3-Small entrenado con SpecAugment.

Los hallazgos más relevantes del trabajo son:

- 1) **La arquitectura importa más que la aumentación para modelos pequeños**: LeNet-5 con SpecAugment (A4, F1=0.7597) rindió peor que LeNet-5 sin aumentar (A1, F1=0.8322), sugiriendo que la capacidad del modelo debe ser suficiente para aprovechar la variabilidad adicional.
- 2) **Las clases auxiliares son indispensables**: sin `_silence_` y `_unknown_`, el sistema produce detecciones espurias continuas en entornos reales.
- 3) **La consistencia del pipeline de preprocesamiento es crítica**: el bug de transposición no fue detectable en pruebas de unidad, sino únicamente al integrar el sistema end-to-end en el dispositivo real.

REFERENCES

- [1] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," *arXiv preprint arXiv:1804.03209*, 2018.
- [2] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition," in *Proc. Interspeech 2019*, pp. 2613–2617, 2019.
- [3] H. Lin, D. Gharbieh, H. Tan, and E. Chng, "Training Keyword Spotters with Limited and Synthesized Speech Data," in *Proc. ICASSP 2020*, pp. 7474–7478, 2020.
- [4] A. Howard et al., "Searching for MobileNetV3," in *Proc. ICCV 2019*, pp. 1314–1324, 2019.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [6] T. Zhang et al., "Hello Edge: Keyword Spotting on Microcontrollers," *arXiv preprint arXiv:1711.07128*, 2017.
- [7] S. Choi et al., "Temporal Convolution for Real-time Keyword Spotting on Mobile Devices," in *Proc. Interspeech 2019*, pp. 3372–3376, 2019.
- [8] D. C. de Andrade, S. Leo, M. L. D. S. Viana, and C. Bernkopf, "A Neural Attention Model for Speech Command Recognition," *arXiv preprint arXiv:1808.08929*, 2018.

- [9] A. Berg, M. O'Connor, and M. T. Cruz, "Keyword Transformer: A Self-Attention Model for Keyword Spotting," in *Proc. Interspeech 2021*, pp. 4249–4253, 2021.
- [10] B. Kim and H. Kim, "Broadcasted Residual Learning for Efficient Keyword Spotting," in *Proc. Interspeech 2021*, pp. 4538–4542, 2021.